

TD7: Ingeniería de Datos

Resumen — Segundo Parcial

Universidad Torcuato Di Tella

8.1 — Calidad y Gobernanza de Datos

Calidad de datos

Definición: aptitud de un dato para ser usado por quien debe consumirlo.

Cómo se mide:

- **Análisis univariado:** mín/máx, media, mediana, moda, cuartiles, histogramas, boxplots
- **Análisis bivariado:** coeficiente de correlación, tablas de contingencia, diagramas de dispersión

8 Dimensiones de calidad

Dimensión	Qué mide
Precisión	Qué tan cercano a la realidad (ej: “Perezz” en vez de “Perez”)
Compleitud	No hay NULLs donde no debería haberlos
Consistencia	Reglas de negocio se mantienen entre tablas
Puntualidad	Disponible cuando se necesita
Validez	Tipo correcto, FK válidas, formato correcto
Unicidad	No hay duplicados del mismo hecho
Conformidad	Formato correcto
Accesibilidad	Disponible para los interesados

Gobernanza de datos

Capacidad organizacional de **asegurar alta calidad** en todos los datos. Implica:

- Definir **responsables** (*data stewards*)
- Políticas de almacenamiento, backup y protección
- Normas de uso y procedimientos de auditoría

Ciclo de vida del dato: Create → Store → Use → Share → Archive → Destroy

Alcanzando la calidad

Pipelines: validar datos en cada pasaje entre subsistemas. En streaming, validar continuamente y definir acción ante cada violación.

Observabilidad: comprensión de salud y desempeño de datos. Técnicas:

- Monitoreo automatizado
- *Root cause analysis* (RCA)
- **Data lineage:** rastreo de cómo se llega a un dato

Objetivo: **prevenir, detectar y resolver anomalías** lo antes posible.

8.2 — Privacidad y Compliance

Roles clave

- **Data Governance Lead:** fija la estrategia de gobernanza de la organización
- **Data Steward:** ejecuta las tareas diarias de gobernanza

PII (Personally Identifiable Information)

Información que **identifica directamente** a un individuo: nombre, dirección, CUIT, teléfono, e-mail.

Las organizaciones deben cumplir normativas de protección (GDPR, CCPA, etc.). La gobernanza de datos implica rastrear dónde vive y fluye PII en todos los pipelines, y controlar quién tiene acceso.

Data Catalogs

Metadatos curados sobre **qué** datos tiene la organización, **quién** los usa y **dónde** están.

Sirven para:

- Gobernanza y compliance
- Rastrear dónde vive y fluye PII
- Saber quién tiene acceso a qué datos
- Responder: ¿dónde busco mis datos? ¿qué representan? ¿son relevantes?

Tipos: in-house (ej: Uber Databook, Airbnb, Netflix), open-source, comerciales.

Data Discovery

Evolución del catálogo: entendimiento **dinámico y en tiempo real** del estado actual de los datos (no el estado “ideal catalogado”). Permite democratización del dato — que todos puedan acceder a los datos que necesitan para su trabajo.

Niveles de madurez:

1. *Data reactive:* nadie accede ni usa datos
2. *Data scaling:* pocos tienen skills para analizar
3. *Data progressive:* al menos un empleado data-fluent por equipo
4. *Data fluent:* todos saben cómo acceder a los datos que necesitan

8.3 — Optimización en SQL Server

Planes de ejecución

SQL Server genera un plan físico visible en el query optimizer. El flujo de datos va de **derecha a izquierda** en el diagrama. El porcentaje en cada operador indica su costo relativo.

Operadores de acceso a datos

Estructura	Scan	Seek
Heap	Table Scan	—
Clustered Index	Clustered Index Scan	Clustered Index Seek
Nonclustered Index	Index Scan	Index Seek

Seek = acceso directo por índice (eficiente). **Scan** = recorre todo (costoso en tablas grandes).

Operadores de agregación

SQL Server usa dos operadores para SUM, AVG, MAX, GROUP BY, DISTINCT:

Stream Aggregate

- *Scalar aggregates* (sin GROUP BY) → siempre Stream Aggregate
- Con GROUP BY → datos deben venir **ordenados** (usa índice o inserta Sort operator)

Hash Aggregate (Hash Match)

- Para tablas grandes con datos **no ordenados**
- No necesita orden; óptimo cuando cardinalidad estimada es pocos grupos

Hash vs Stream

Si se crea un índice que provee datos ordenados, el optimizador puede elegir Stream en vez de Hash. Si la entrada no está ordenada pero se pide orden explícito, el optimizador puede: insertar Sort + Stream Aggregate, o usar Hash Aggregate + Sort al final.

Distinct Sort

DISTINCT puede implementarse con Stream Aggregate, Hash Aggregate o **Distinct Sort**.

Juntas (Joins)

El optimizador debe decidir: **orden de las juntas y algoritmo de junta**.

Algoritmo	Inner table	Outer table	Tamaño	Ordenado	Cláusula
Hash Join	Sin índice	Opcional	Cualquiera	No	Equi-join
Merge Join	Con índice	Con índice	Grande	Sí	Equi-join
Nested Loops	Con índice (must)	Preferable	Pequeño	Opcional	Cualquiera

Notas clave:

- **Nested Loops:** ideal outer pequeño + inner grande indexado. Acepta cualquier predicado de junta.
- **Merge Join:** ambas tablas pre-ordenadas; óptimo con índice clustered o covering en ambas.
- **Hash Join:** no requiere orden; óptimo outer pequeño + inner grande. Implementado con Hash Match.
- Merge y Hash **requieren equi-join**; Nested Loops acepta cualquier predicado.